

Pymetory

Estudio de costos de operación del asistente

Germán David Murillas Mondragón · Universidad del Valle, sede Tuluá · Anexo digital del trabajo de grado

Mediciones y precios del 1 de octubre de 2026

1. Propósito

Pymetory se puede instalar sin licencias pagas: el servidor usa la capa gratuita de Oracle Cloud y el código es propio. El único costo que crece con el uso es el del modelo de lenguaje del asistente, porque los proveedores cobran por cada consulta según la cantidad de texto que reciben y que generan. Antes de ofrecer el sistema a otra PYME hay que saber cuánto cuesta ese uso y de qué depende.

Este estudio responde tres preguntas con datos medidos y no con supuestos: cuánto texto procesa el modelo en una consulta típica, cuánto cuesta esa consulta con distintos proveedores, y qué opción conviene para una empresa del tamaño de la panificadora del caso de estudio. Al final se propone una prueba piloto para medir cuántas consultas hace la empresa en un mes real, que es el dato que todavía falta para fijar una tarifa.

2. Cómo cobran los proveedores

Los modelos de lenguaje no leen palabras sino *tokens*: fragmentos de texto de unos cuatro caracteres en promedio. Cada consulta tiene dos partes que se cobran por separado:

- **Tokens de entrada:** las instrucciones del sistema, los datos del inventario que el RAG adjunta y la pregunta del usuario.
- **Tokens de salida:** la respuesta que redacta el modelo. Suelen costar entre cuatro y cinco veces más que los de entrada.

Los precios se publican en dólares por millón de tokens. Hay dos formas de pagar: **por uso** (se descuenta de un saldo prepagado según lo consumido) o por **suscripción** mensual con un límite de uso. Más adelante se comparan ambas.

3. Cómo se midió

Cada proveedor informa en su respuesta cuántos tokens procesó (campo `usage` en las API compatibles con OpenAI y `prompt_eval_count/eval_count` en Ollama). Desde el commit `6ddad01` Pymetory guarda esos dos valores y el nombre del modelo en cada registro de `chat_histories`, y el comando de evaluación `php artisan rag:evaluar` los resume.

El 1 de octubre de 2026 se ejecutaron en producción las cinco baterías de preguntas usadas en el Capítulo 11 (106 preguntas: batería de 50, validación de 20, ciega de 10, operario coloquial de 18 y operario ciega de 8), con el modelo que usa el sistema, `gpt-oss:120b` en Ollama Cloud. Las mediciones de este documento son las de la segunda ejecución de ese día, con la configuración vigente (sin tope de salida para los modelos en la nube, ver más abajo); las 106 respuestas reportaron su consumo y todas las dio `gpt-oss:120b`. Los conteos dependen del tokenizador de cada modelo, así que con otro modelo las cifras pueden variar algo; el orden de magnitud se mantiene.

4. Consumo medido

	Promedio	Mediana	Percentil 95	Mínimo	Máximo
Tokens de entrada	687	525	1 248	464	1 259
Tokens de salida	333	179	1 270	49	1 410

Tabla 1: Tokens por consulta en las 106 preguntas (1-oct-2026, `gpt-oss:120b`).

El consumo depende del tipo de pregunta. Las consultas puntuales (existencias, ubicación, valor de un insumo) usan poco, porque el RAG solo adjunta los lotes del insumo nombrado. Las que piden listas (lotes por vencer, alertas) usan más, porque la respuesta enumera muchos lotes:

Tipo de pregunta	Preguntas	Entrada (prom.)	Salida (prom.)
Vencimiento por periodo (“¿qué vence esta semana?”)	2	1 230	928
Lotes críticos (“¿qué está por vencer?”)	14	1 196	1 110
Movimientos del Kardex	8	964	284
Resumen general	4	941	428
Stock bajo el mínimo	5	769	422
Información de un lote	7	691	372
Valorización	9	536	101
Insumo que no existe	11	531	208
Existencias	19	511	84
Conciliación	4	503	247
Vencimiento de un insumo	8	488	168
Ubicación	9	484	122
Cuarentena	6	468	165

Tabla 2: Consumo promedio por tipo de pregunta.

Lo que ahorra el RAG. Para comparar, se midió con el mismo tokenizador cuánto ocuparía enviar al modelo *todo* el inventario en cada consulta (los 58 lotes con existencias de los 45 insumos, en el mismo formato que usa el RAG): **3 467 tokens de entrada**, frente a una mediana de 525 con el RAG, es decir, unas 6,6 veces más. Esa cifra todavía no incluye los 2 069 movimientos del Kardex, y crecería con cada lote nuevo, mientras que el contexto del RAG depende de la pregunta y no del tamaño del inventario. Por eso el RAG no solo mejora la precisión: también es lo que mantiene bajo el costo cuando la empresa crece.

Un hallazgo de la medición. En la primera ejecución del 1 de octubre la salida máxima fue exactamente 1 024 tokens, el límite que estaba configurado en Ajustes (*Max Tokens*). Las respuestas que llegaban a ese tope eran las listas largas de lotes por vencer, y el modelo las cortaba a mitad de la lista: la batería de 50 preguntas bajó a 45/50 y la de operario a 11/18. Desde el commit `47cf3c6` el límite solo se aplica a los modelos que corren en el propio servidor, donde una respuesta larga puede tardar minutos o saturar la memoria; los modelos en la nube responden completo. En la segunda ejecución la batería volvió a 49/50 y la de operario a 18/18, con una salida máxima de 1 410 tokens. Quitar el tope sube el costo de las consultas de listas, pero muy poco (tabla siguiente).

5. Costo por consulta

El costo de una consulta es

$$\text{costo} = \frac{\text{tokens de entrada} \times \text{precio de entrada} + \text{tokens de salida} \times \text{precio de salida}}{1\,000\,000}.$$

La tabla aplica el consumo medido a los precios oficiales de cada proveedor [1, 2, 3, 4], convertidos con la TRM del 1 de octubre de 2026, \$3 312,84 por dólar [6]. Solo `gpt-oss:120b` fue evaluado en precisión con Pymetory; los demás se incluyen como referencia de precio.

Modelo (proveedor)	USD por millón		COP por consulta promedio	COP por consulta (p95)
	Entrada	Salida		
<code>gpt-oss:120b</code> (Ollama Cloud) — el actual	0,15	0,60	1,00	3,14
DeepSeek V4.1 Flash, horario valle (DeepSeek)	0,15	0,60	1,00	3,14
DeepSeek V4.1 Flash, hora pico (DeepSeek)	0,30	1,20	2,01	6,29
Gemini 2.5 Flash-Lite, nivel pago (Google)	0,10	0,40	0,67	2,10
Gemini 3.1 Flash-Lite, nivel pago (Google)	0,25	1,50	2,22	7,34
GLM-5.3-Flash (OpenCode Zen)	0,15	0,50	0,89	2,72
Qwen3.8 Flash (OpenCode Zen)	0,15	0,47	0,86	2,60
GPT 5 Nano (OpenCode Zen)	0,05	0,40	0,56	1,89
Claude Haiku 4.5 (OpenCode Zen)	1,00	5,00	7,79	25,17
Kimi K3 (OpenCode Zen)	3,00	15,00	23,38	75,51

Tabla 3: Costo por consulta con el consumo medido (promedio: 687 de entrada y 333 de salida; p95: 1 248 y 1 270).

Con los modelos económicos una consulta cuesta alrededor de **un peso colombiano**. Los modelos de gama alta (Claude Haiku, Kimi K3) cuestan entre 8 y 23 veces más sin que la tarea lo exija: el modelo no calcula ni busca datos, solo redacta a partir de lo que el sistema ya consultó en la base de datos.

6. Proyección mensual

Todavía no se sabe cuántas consultas hace la panificadora al día; ese es el objetivo de la prueba piloto (sección 13). Mientras tanto, la tabla muestra tres escenarios *hipotéticos* de 30 días, con el costo promedio por consulta:

Modelo	20 consultas/día	50 consultas/día	100 consultas/día
<code>gpt-oss:120b</code> (actual)	\$602	\$1 505	\$3 010
GPT 5 Nano	\$333	\$833	\$1 665
Gemini 2.5 Flash-Lite (pago)	\$401	\$1 003	\$2 007
Claude Haiku 4.5	\$4 675	\$11 688	\$23 375

Tabla 4: Costo mensual en pesos colombianos según el número de consultas (escenarios, no mediciones).

Aun en el escenario alto, el modelo actual cuesta menos de un dólar al mes. El costo del asistente no es una barrera para una PYME; pesan más el soporte y el tiempo de capacitación, que este estudio no cuantifica.

7. Suscripción o pago por uso

OpenCode Go. Cuesta US\$10 al mes (Go Plus, US\$40). El uso se mide en dólares por modelo: cada modelo tiene un tope mensual (por ejemplo, US\$60 para GLM-5.3-Flash en Go), del que se puede gastar el 20 % en cinco horas y el 50 % en una semana [5]. Con el consumo medido, los US\$10 equivalen a unas 37 000 consultas con GLM-5.3-Flash, alrededor de 1 240 al día: mucho más de lo que necesita una panadería. Pagar por uso sale más barato a esta escala.

Hay además una restricción de uso. La documentación dice: “*OpenCode Go is designed for OpenCode and other coding agents that produce similar types of requests*” y pide a los clientes “*send typical coding agent traffic*” [5]. Un asistente de inventarios no es un agente de programación, así que OpenCode Go solo se usó para pruebas durante el desarrollo y **no se recomienda para operar el sistema con clientes**. Pymetory se identifica ante ese servicio con su propio nombre (`pymetory/1.0`).

Ollama Cloud. Es lo que usa hoy el sistema. El plan gratuito incluye un monto inicial de uso para un grupo de modelos y atiende una petición a la vez; Ollama no publica de cuánto es ese monto [1]. El plan Pro cuesta US\$20 al mes con US\$60 de crédito y tres peticiones simultáneas. Para la escala de una panadería, comprar créditos y pagar por uso es suficiente; el plan Pro solo se justificaría con varios usuarios consultando a la vez.

Capas gratuitas. Gemini 2.5 Flash-Lite tiene capa gratuita, pero Google indica que en ella los datos *se usan para mejorar sus productos* (en el nivel pago, no) [3]. Sirve para un piloto con datos de demostración, no para los datos reales de una empresa.

8. Precisión por modelo

Para saber si un modelo más barato o más costoso cambia la calidad de las respuestas, el 1 de octubre de 2026 se repitieron las 106 preguntas con nueve modelos más, todos con el mismo RAG y los mismos datos de producción. Se usó la opción `rag:evaluar -fuente -modelo`, que cambia el modelo solo durante la medición. Los modelos de OpenCode Go se llamaron con la suscripción del autor, como prueba de desarrollo. Las respuestas que no dio el modelo evaluado (por ejemplo, del respaldo local) no se le contaron como aciertos.

El criterio automático marca como error cualquier respuesta a un insumo inexistente que incluya una cantidad. Varios modelos respondieron bien (“no hay chocolate blanco registrado”) y añadieron la

cantidad de un producto parecido (“solo figura pan tajado blanco: 811 und”). Por eso la tabla trae también la cifra con revisión manual de esas respuestas. El criterio automático no se modificó después de ver los resultados.

Modelo	Automático	Revisión manual	Tiempo mediano	COP por consulta
<code>gpt-oss:120b</code> (actual) ^a	101	102	0,70 s	1,00
DeepSeek V4 Pro	105	105	4,70 s	4,28 (8,56)
GLM-5.3	101	104	4,01 s	8,32
Kimi K3	100	103	6,79 s	27,22
DeepSeek V4.1 Flash	100	103	2,24 s	0,88 (1,76)
GLM-5.3-Flash	102	104	3,08 s	0,84
Qwen3.8 Flash	102	102	4,28 s	1,06
MiMo-V2.6-Flash	100	103	4,51 s	0,57
Space Bunny Free ^b	102	102	2,62 s	0
LongCat 2.5 Preview Free ^b	100	101	5,30 s	0

Tabla 5: Aciertos sobre 106 preguntas por modelo, con el mismo RAG (1-oct-2026, una ejecución por modelo). (a) En Ollama Cloud; los demás, en OpenCode Go. (b) Gratis por tiempo limitado. Entre paréntesis, el costo en hora pico de DeepSeek.

Todos los modelos quedan entre 100 y 105 aciertos. Con un RAG que entrega los datos ya filtrados, la precisión casi no depende del modelo: un modelo económico o gratuito responde tan bien como uno de gama alta, que cuesta hasta 27 veces más y tarda hasta diez veces más. Las diferencias de una a tres preguntas no son concluyentes con una sola ejecución por modelo, porque las respuestas varían algo entre ejecuciones. Solo DeepSeek V4 Pro se distingue: no omitió datos que los demás sí omitieron, a cambio de unos 5 segundos por respuesta y cuatro a ocho veces el costo. Dos modelos (Muse Spark y GPT 6 Luna) no se pudieron probar porque usan otro formato de API.

Todos los modelos fallaron la misma pregunta: “¿Qué se está poniendo viejo y hay que sacar ya?”. El error no era del modelo sino del clasificador de intención, que la tomó como una pregunta de existencias.

9. Clasificador y respaldo en la nube

Con ese resultado se hicieron dos cambios (commits `0afef8a` y `0d8f5eb`):

- **El modelo clasifica primero.** Antes decidían los patrones y el modelo solo intervenía si no reconocían la pregunta. Ahora un número de lote se reconoce siempre de forma exacta; para lo demás decide el modelo (`gpt-oss:120b`), y los patrones quedan como red de seguridad cuando el modelo no responde o contesta algo fuera de las categorías. La descripción de las categorías no se cambió, para no ajustarla a las preguntas de las baterías ciegas.
- **Respaldo en la nube.** Si el modelo principal falla, responde primero un modelo rápido en la nube (en las pruebas, GLM-5.3-Flash por OpenCode Go) y solo después el modelo local. En una prueba con el modelo principal caído, la respuesta llegó en 2,0 segundos, frente a 56,4 segundos del modelo local.

Con el clasificador nuevo, `gpt-oss:120b` pasó de 101 a 103 aciertos (50/50, 20/20, 8/10, 18/18 y 7/8) y la pregunta del clasificador quedó bien resuelta. Los tres errores que quedan son omisiones del modelo con los datos a la vista. El costo es una llamada más por consulta: la entrada promedio sube de 687 a 997 tokens, la salida de 333 a 406, el tiempo mediano de 0,72 a 1,17 segundos y el costo de unos 1,00 a unos 1,30 pesos por consulta.

10. Palanca de modelos en Pymetory

En **Ajustes** → **Motor RAG**, el administrador elige el modelo con una palanca que ordena los modelos de menor a mayor costo por consulta, con un degradado de azul (económico) a rojo (costoso), y con verde en el modelo recomendado. En cada posición se ven el nombre del modelo, la precisión medida, el tiempo de respuesta, los tokens por consulta y el costo frente al recomendado en porcentaje. Los datos salen de la base de datos: cada ejecución de `rag:evaluar` se guarda en la tabla `llm_evaluaciones`, y los precios de esta sección están en `llm_precios`, con su fuente y fecha de consulta. Así el usuario sabe cuánto consume cada opción antes de elegirla.

11. Recomendación

1. **Mantener gpt-oss:120b con pago por uso.** Es el más rápido de los diez medidos (menos de un segundo), su precisión está en el mismo rango que los demás y cuesta alrededor de un peso por consulta.
2. **Alternativas de costo similar o menor:** GLM-5.3-Flash, DeepSeek V4.1 Flash, MiMo-V2.6-Flash o Qwen3.8 Flash (medidos en la tabla de precisión), o GPT 5 Nano y Gemini 2.5 Flash-Lite de pago (no medidos). Antes de cambiar a un modelo no medido hay que repetir las baterías con `rag:evaluar`.
3. **Usar un modelo rápido en la nube como respaldo**, antes del modelo local. En estas pruebas se configuró GLM-5.3-Flash por OpenCode Go; para operar con clientes, el mismo respaldo se configura con un proveedor de pago por uso (ajustes `llm_respaldo_fuente` y `llm_respaldo_modelo`).
4. **No usar modelos de gama alta** para esta tarea: multiplican el costo sin una mejora necesaria.
5. **No operar con OpenCode Go** ni con capas gratuitas que usan los datos para entrenar.

12. Cómo configurar una API key en Pymetory

Solo el administrador puede hacerlo.

1. Crear la clave en el sitio del proveedor y cargar saldo si el proveedor lo exige.
2. En Pymetory, abrir **Ajustes** → **API Keys**. Escribir un nombre, elegir el proveedor (OpenCode, OpenCode Go, Ollama u OpenAI), pegar la clave y elegir el modelo. La dirección del servicio se completa sola según el tipo.
3. Guardar con **Nueva Key**. La clave se guarda cifrada en la base de datos y en la lista solo se ven sus primeros y últimos cuatro caracteres. Solo puede haber una clave activa por tipo.
4. Pulsar **Probar conexión** junto a la clave: el sistema envía un mensaje corto al proveedor e informa si respondió.
5. En **Ajustes** → **Motor RAG**, elegir el *Origen* y el *Modelo*, y guardar. *Max Tokens* solo limita los modelos que corren en el propio servidor; los modelos en la nube responden sin tope.

13. Prueba piloto propuesta

El costo mensual real depende de cuántas consultas hace la empresa, y eso solo se sabe usando el sistema en su operación diaria. La propuesta es dejar el asistente en uso normal en la panificadora durante cuatro semanas y luego consultar el consumo, excluyendo las consultas de prueba hechas por el desarrollador:

```
SELECT DATE(created_at) AS dia, COUNT(*) AS consultas,
```

```

SUM(tokens_entrada) AS entrada, SUM(tokens_salida) AS salida
FROM chat_histories
WHERE tokens_entrada IS NOT NULL AND user_id <> <id del administrador de pruebas>
AND created_at >= '<fecha de inicio del piloto>'
GROUP BY dia ORDER BY dia;

```

Con esos totales y los precios de este documento se obtiene el costo real por día y por mes, y con varias empresas de distinto tamaño se puede fijar una tarifa con un margen sobre ese costo. La sesión de prueba con la usuaria (Capítulo 11) es el primer punto de medición.

14. Limitaciones

- Los precios cambian con frecuencia; los de este documento son los publicados el 1 de octubre de 2026. DeepSeek cobra el doble en hora pico (01:00–04:00 y 06:00–10:00 UTC, de lunes a viernes) y Google anunció que Gemini 3.x Flash sube de precio en 2027.
- Las 106 preguntas son de prueba, no de uso real; por eso la proyección mensual se presenta como escenarios.
- Cada modelo se midió una sola vez; las diferencias de una a tres preguntas pueden deberse a la variación entre ejecuciones. La comparación entre modelos se hizo con el clasificador anterior (por patrones). GPT 5 Nano y los modelos de Gemini no se midieron.
- No se incluyen costos de servidor (capa gratuita de Oracle Cloud), dominio ni soporte.

Referencias

- [1] Ollama, “Pricing,” 2026. [En línea]. Disponible: <https://ollama.com/pricing>. [Accedido: 1-oct-2026].
- [2] DeepSeek, “Models & Pricing,” 2026. [En línea]. Disponible: https://api-docs.deepseek.com/quick_start/pricing. [Accedido: 1-oct-2026].
- [3] Google, “Gemini Developer API pricing,” 2026. [En línea]. Disponible: <https://ai.google.dev/gemini-api/docs/pricing>. [Accedido: 1-oct-2026].
- [4] OpenCode, “Zen,” 2026. [En línea]. Disponible: <https://opencode.ai/docs/zen/>. [Accedido: 1-oct-2026].
- [5] OpenCode, “Go,” 2026. [En línea]. Disponible: <https://opencode.ai/docs/go/>. [Accedido: 1-oct-2026].
- [6] Superintendencia Financiera de Colombia, “Tasa de Cambio Representativa del Mercado (TRM),” Datos Abiertos Colombia, 2026. [En línea]. Disponible: <https://www.datos.gov.co/resource/32sa-8pi3>. [Accedido: 1-oct-2026].